MATERI 6
Local Storage & Offline First

1. Buka proyek yang sudah anta buka, bisa melalui **vscode** atau **android studio**
2. Ubah pada menu ***dosen*** dengan menggunakan data API berikut menggunakan http ***https://jsonplaceholder.typicode.com/users***
3. Tambahkan **dependencies**

```
dependencies:
  flutter:
    sdk: flutter
  flutter_riverpod: ^2.6.1
  http: ^1.2.2
  dio: ^5.8.0+1 // tambahkan ini
  shared_preferences: ^2.5.3 // tambahkan ini
```

4. Buat file baru ada di ***core/services/local_storage_service.dart*** berfungsi sebagai class untuk penyimpanan dilocal

```dart
import 'package:shared_preferences/shared_preferences.dart';
import 'dart:convert';

class LocalStorageService {
  static const String _tokenKey = 'auth_token';
  static const String _userIdKey = 'user_id';
  static const String _usernameKey = 'username';
  static const String _savedUsersKey = 'saved_users';

  // ── Token ──────────────────────────────────────
  Future<void> saveToken(String token) async {
    final prefs = await SharedPreferences.getInstance();
    await prefs.setString(_tokenKey, token);
  }

  Future<String?> getToken() async {
    final prefs = await SharedPreferences.getInstance();
    return prefs.getString(_tokenKey);
  }

  Future<void> removeToken() async {
    final prefs = await SharedPreferences.getInstance();
    await prefs.remove(_tokenKey);
  }

  // ── User ───────────────────────────────────────
  Future<void> saveUser({
    required String userId,
    required String username,
  }) async {
    final prefs = await SharedPreferences.getInstance();
    await prefs.setString(_userIdKey, userId);
    await prefs.setString(_usernameKey, username);
  }

  Future<String?> getUserId() async {
    final prefs = await SharedPreferences.getInstance();
    return prefs.getString(_userIdKey);
  }

  Future<String?> getUsername() async {
    final prefs = await SharedPreferences.getInstance();
    return prefs.getString(_usernameKey);
  }
```

```dart
// ── simpan user di list ──────────────────────
/// Tambah user ke list (tidak menghapus yang lama)
Future<void> addUserToSavedList({
  required String userId,
  required String username,
}) async {
  final prefs = await SharedPreferences.getInstance();
  final rawList = prefs.getStringList(_savedUsersKey) ?? [];

  // Cek duplikasi userId
  final isDuplicate = rawList.any((item) {
    final map = jsonDecode(item) as Map<String, dynamic>;
    return map['user_id'] == userId;
  });

  if (isDuplicate) return; // Jika sudah ada, lewati

  final newUser = jsonEncode({
    'user_id': userId,
    'username': username,
    'saved_at': DateTime.now().toIso8601String(),
  });

  rawList.add(newUser);
  await prefs.setStringList(_savedUsersKey, rawList);
}

/// Ambil semua user yang sudah disimpan
Future<List<Map<String, String>>> getSavedUsers() async {
  final prefs = await SharedPreferences.getInstance();
  final rawList = prefs.getStringList(_savedUsersKey) ?? [];

  return rawList.map((item) {
    final map = jsonDecode(item) as Map<String, dynamic>;
    return {
      'user_id': (map['user_id'] ?? '').toString(),
      'username': (map['username'] ?? '').toString(),
      'saved_at': (map['saved_at'] ?? '').toString(),
    };
  }).toList();
}
```

```dart
/// delete user tertentu dari list
Future<void> removeSavedUser(String userId) async {
  final prefs = await SharedPreferences.getInstance();
  final rawList = prefs.getStringList(_savedUsersKey) ?? [];

  rawList.removeWhere((item) {
    final map = jsonDecode(item) as Map<String, dynamic>;
    return map['user_id'] == userId;
  });

  await prefs.setStringList(_savedUsersKey, rawList);
}

/// delete semua user dari list
Future<void> clearSavedUsers() async {
  final prefs = await SharedPreferences.getInstance();
  await prefs.remove(_savedUsersKey);
}

// ── Clear All preferences──────────────────────────
Future<void> clearAll() async {
  final prefs = await SharedPreferences.getInstance();
  await prefs.clear();
}
}
```

5. Buat file baru ada di ***core/network/dio_client.dart*** berfungsi sebagai class untuk pengaturan api secara terpusat dengan menggunakan dio.

```dart
import 'package:d4tivokasi/core/services/local_storage_service.dart';
import 'package:dio/dio.dart';

class DioClient {
  static const String _baseUrl = 'https://jsonplaceholder.typicode.com';

  final Dio _dio;
  final LocalStorageService _localStorage;

  DioClient({LocalStorageService? localStorage})
      : _localStorage = localStorage ?? LocalStorageService(),
        _dio = Dio(
          BaseOptions(
            baseUrl: _baseUrl,
            connectTimeout: const Duration(seconds: 15),
            receiveTimeout: const Duration(seconds: 15),
            headers: {'Accept': 'application/json'},
          ), // BaseOptions
        ) { // Dio
    _dio.interceptors.add(
      InterceptorsWrapper(
        onRequest: (options, handler) async {
          // Otomatis sisipkan token dari SharedPreferences
          final token = await _localStorage.getToken();
          if (token != null) {
            options.headers['Authorization'] = 'Bearer $token';
          }
          handler.next(options);
        },
        onResponse: (response, handler) {
          handler.next(response);
        },
        onError: (DioException error, handler) {
          handler.next(error);
        },
      ),
    );
  }

  Dio get dio => _dio;
}
```

6. Ubah **dosen_repository.dart**

```dart
import 'package:d4tivokasi/core/network/dio_client.dart';
import 'package:d4tivokasi/features/dosen/data/models/dosen_model.dart';
import 'package:dio/dio.dart';

class DosenRepository {
  final DioClient _dioClient;

  DosenRepository({DioClient? dioClient})
    : _dioClient = dioClient ?? DioClient();

  /// get data daftar dosen
  Future<List<DosenModel>> getDosenList() async {
    try {
      final Response response = await _dioClient.dio.get('/users');
      final List<dynamic> data = response.data;
      return data.map((json) => DosenModel.fromJson(json)).toList();
    } on DioException catch (e) {
      throw Exception(
        'Gagal memuat data dosen: ${e.response?.statusCode} - ${e.message}',
      );
    }
  }
}
```

7. Ubah **dosen_provider.dart**

```dart
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:d4tivokasi/core/services/local_storage_service.dart';
import 'package:d4tivokasi/features/dosen/data/models/dosen_model.dart';
import 'package:d4tivokasi/features/dosen/data/repositories/dosen_repository.dart';

// Repository Provider
final dosenRepositoryProvider = Provider<DosenRepository>((ref) {
  return DosenRepository();
}); // Provider

// LocalStorageService Provider
final localStorageServiceProvider = Provider<LocalStorageService>((ref) {
  return LocalStorageService();
}); // Provider

// —— Provider semua data user yang disimpan ——————————————
final savedUsersProvider = FutureProvider<List<Map<String, String>>>((
  ref,
) async {
  final storage = ref.watch(localStorageServiceProvider);
  return storage.getSavedUsers();
});

// Provider untuk membaca saved user dari local storage
final savedUserProvider = FutureProvider<Map<String, String?>>((ref) async {
  final storage = ref.watch(localStorageServiceProvider);
  final userId = await storage.getUserId();
  final username = await storage.getUsername();
  final token = await storage.getToken();
  return {'userId': userId, 'username': username, 'token': token};
});
```

```dart
// StateNotifier untuk mengelola state dosen
class DosenNotifier extends StateNotifier<AsyncValue<List<DosenModel>>> {
  final DosenRepository _repository;
  final LocalStorageService _storage;

  DosenNotifier(this._repository, this._storage)
    : super(const AsyncValue.loading()) {
    loadDosenList();
  }

  Future<void> loadDosenList() async {
    state = const AsyncValue.loading();
    try {
      final data = await _repository.getDosenList();
      state = AsyncValue.data(data);
    } catch (error, stackTrace) {
      state = AsyncValue.error(error, stackTrace);
    }
  }

  Future<void> refresh() async {
    await loadDosenList();
  }

  /// Simpan dosen yang dipilih ke local storage (tanpa menghapus yang lama)
  Future<void> saveSelectedDosen(DosenModel dosen) async {
    await _storage.addUserToSavedList(
      userId: dosen.id.toString(),
      username: dosen.username,
    );
  }

  /// Hapus user tertentu dari list
  Future<void> removeSavedUser(String userId) async {
    await _storage.removeSavedUser(userId);
  }

  /// Hapus semua user dari list
  Future<void> clearSavedUsers() async {
    await _storage.clearSavedUsers();
  }
}
```

```dart
// Dosen Notifier Provider
final dosenNotifierProvider =
    StateNotifierProvider.autoDispose<
      DosenNotifier,
      AsyncValue<List<DosenModel>>
    >((ref) {
      final repository = ref.watch(dosenRepositoryProvider);
      final storage = ref.watch(localStorageServiceProvider);
      return DosenNotifier(repository, storage);
    });
```

8. Ubah **dosen_page.dart**

```dart
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:d4tivokasi/core/widgets/widgets.dart';
import 'package:d4tivokasi/features/dosen/data/models/dosen_model.dart';
import 'package:d4tivokasi/features/dosen/presentation/providers/dosen_provider.dart';
import 'package:d4tivokasi/features/dosen/presentation/widgets/dosen_widget.dart';

class DosenPage extends ConsumerWidget {
  const DosenPage({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final dosenState = ref.watch(dosenNotifierProvider);
    final savedUsers = ref.watch(savedUsersProvider); // ← CHANGED

    return Scaffold(
      appBar: AppBar(
        title: const Text('Data Dosen'),
        elevation: 0,
        actions: [
          IconButton(
            icon: const Icon(Icons.refresh_rounded),
            onPressed: () => ref.invalidate(dosenNotifierProvider),
            tooltip: 'Refresh',
          ), // IconButton
        ],
      ), // AppBar
      body: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          // ── Section: Data Tersimpan di SharedPreferences ──
          _SavedUserSection(savedUsers: savedUsers, ref: ref),

          // ── Section Title: Daftar Dosen ──────────────────
          const Padding(
            padding: EdgeInsets.fromLTRB(16, 8, 16, 4),
            child: Text(
              'Daftar Dosen',
              style: TextStyle(fontSize: 14, fontWeight: FontWeight.bold),
            ), // Text
          ), // Padding

          // ── Dosen List ────────────────────────────────────
          Expanded(
            child: dosenState.when(
              loading: () => const LoadingWidget(),
              error: (error, stack) => CustomErrorWidget(
                message: 'Gagal memuat data dosen: ${error.toString()}',
                onRetry: () {
                  ref.read(dosenNotifierProvider.notifier).refresh();
                },
              ), // CustomErrorWidget
              data: (dosenList) => _DosenListWithSave(
                dosenList: dosenList,
                onRefresh: () => ref.invalidate(dosenNotifierProvider),
              ), // _DosenListWithSave
            ),
          ), // Expanded
        ],
      ), // Column
    ); // Scaffold
  }
}
```

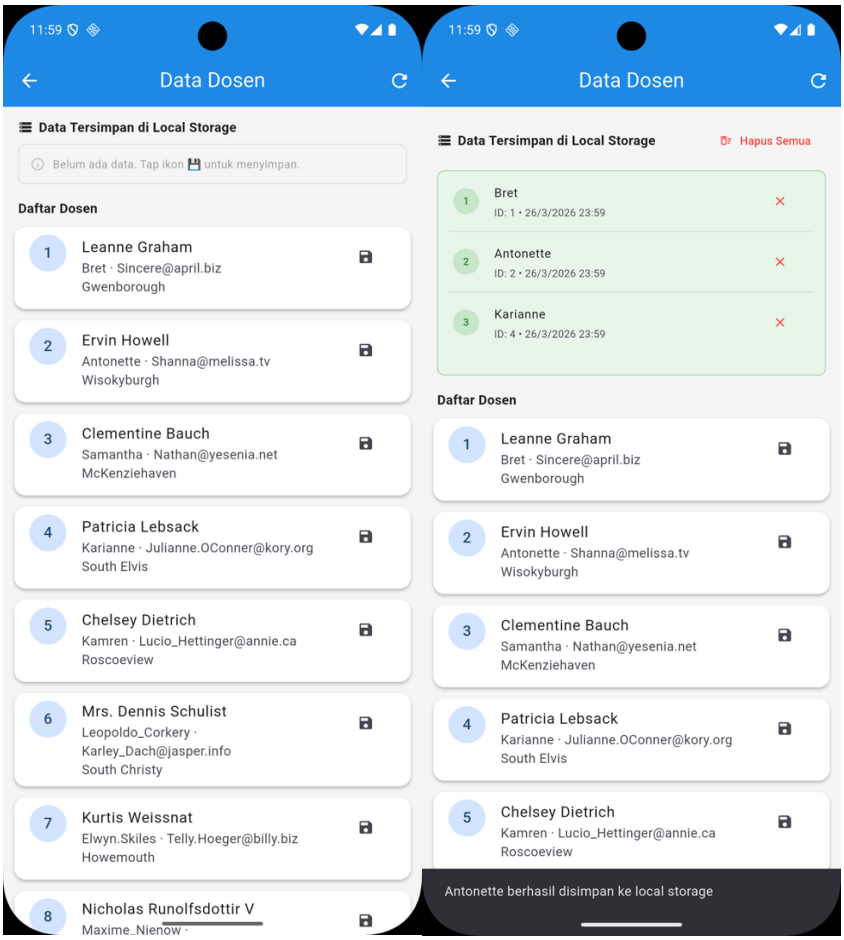
```
// ── Widget: Section data SharedPreferences ──────────────
class _SavedUserSection extends ConsumerWidget {
  final AsyncValue<List<Map<String, String>>> savedUsers;
  final WidgetRef ref;

  const _SavedUserSection({required this.savedUsers, required this.ref});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    return Padding(
      padding: const EdgeInsets.fromLTRB(16, 12, 16, 8),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          // Header dengan tombol hapus semua
          Row(
            children: [
              const Icon(Icons.storage_rounded, size: 16),
              const SizedBox(width: 6),
              const Text(
                'Data Tersimpan di Local Storage',
                style: TextStyle(fontSize: 14, fontWeight: FontWeight.bold),
              ), // Text
              const Spacer(),
              savedUsers.maybeWhen(
                data: (users) => users.isNotEmpty
                    ? TextButton.icon(
                        onPressed: () async {
                          await ref
                              .read(dosenNotifierProvider.notifier)
                              .clearSavedUsers();
                          ref.invalidate(savedUsersProvider);
                          if (context.mounted) {
                            ScaffoldMessenger.of(context).showSnackBar(
                              const SnackBar(
                                content: Text('Semua data tersimpan dihapus'),
                              ), // SnackBar
                            );
                          }
                        },
                        icon: const Icon(
                          Icons.delete_sweep_outlined,
                          size: 14,
                          color: Colors.red,
                        ), // Icon
                        label: const Text(
                          'Hapus Semua',
                          style: TextStyle(fontSize: 12, color: Colors.red),
                        ), // Text
                      ) // TextButton.icon
                    : const SizedBox.shrink(),
                orElse: () => const SizedBox.shrink(),
              ),
            ],
          ), // Row
          const SizedBox(height: 8),

          // Content
```

```
// Content
savedUsers.when(
  loading: () => const LinearProgressIndicator(),
  error: (_, __) => const Text(
    'Gagal membaca data tersimpan',
    style: TextStyle(color: Colors.red, fontSize: 12),
  ), // Text
  data: (users) {
    if (users.isEmpty) {
      return Container(
        width: double.infinity,
        padding: const EdgeInsets.all(12),
        decoration: BoxDecoration(
          color: Colors.grey.shade100,
          borderRadius: BorderRadius.circular(8),
          border: Border.all(color: Colors.grey.shade300),
        ), // BoxDecoration
        child: Row(
          children: [
            Icon(
              Icons.info_outline,
              size: 16,
              color: Colors.grey.shade400,
            ), // Icon
            const SizedBox(width: 8),
            Text(
              'Belum ada data. Tap ikon 💾 untuk menyimpan.',
              style: TextStyle(
                fontSize: 12,
                color: Colors.grey.shade500,
              ), // TextStyle
            ), // Text
          ],
        ), // Row
      ); // Container
    }

    return Container(
      decoration: BoxDecoration(
        color: Colors.green.shade50,
        borderRadius: BorderRadius.circular(8),
        border: Border.all(color: Colors.green.shade200),
      ), // BoxDecoration
      child: ListView.separated(
        shrinkWrap: true,
        physics: const NeverScrollableScrollPhysics(),
        itemCount: users.length,
        separatorBuilder: (_, __) => Divider(
          height: 1,
          color: Colors.green.shade100,
          indent: 12,
          endIndent: 12,
        ), // Divider
        itemBuilder: (context, index) {
          final user = users[index];
          return ListTile(
            dense: true,
            leading: CircleAvatar(
              radius: 14,
              backgroundColor: Colors.green.shade100,
              child: Text(
                '${index + 1}',
                style: TextStyle(
                  fontSize: 11,
                  color: Colors.green.shade700,
                  fontWeight: FontWeight.bold,
```

```
                    color: Colors.green.shade700,
                    fontWeight: FontWeight.bold,
                  ), // TextStyle
                ), // Text
              ), // CircleAvatar
              title: Text(user['username'] ?? '-'),
              subtitle: Text(
                'ID: ${user['user_id']} • ${_formatDate(user['saved_at'] ?? '')}',
                style: const TextStyle(fontSize: 11),
              ), // Text
              trailing: IconButton(
                icon: const Icon(
                  Icons.close,
                  size: 16,
                  color: Colors.red,
                ), // Icon
                onPressed: () async {
                  await ref
                      .read(dosenNotifierProvider.notifier)
                      .removeSavedUser(user['user_id'] ?? '');
                  ref.invalidate(savedUsersProvider);
                  if (context.mounted) {
                    ScaffoldMessenger.of(context).showSnackBar(
                      SnackBar(
                        content: Text('${user['username']} dihapus'),
                      ), // SnackBar
                    );
                  }
                },
              ), // IconButton
            ); // ListTile
          },
        ), // ListView.separated
      ); // Container
    },
  ),
],
), // Column
); // Padding
}

String _formatDate(String isoString) {
  if (isoString.isEmpty) return '-';
  try {
    final date = DateTime.parse(isoString);
    return '${date.day}/${date.month}/${date.year} ${date.hour}:${date.minute}';
  } catch (e) {
    return isoString;
  }
}
}
```

```
// ── Widget: List dosen dengan tombol save ────────────────────
class _DosenListWithSave extends ConsumerWidget {
  final List<DosenModel> dosenList;
  final VoidCallback onRefresh;

  const _DosenListWithSave({required this.dosenList, required this.onRefresh});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    return RefreshIndicator(
      onRefresh: () async => onRefresh(),
      child: ListView.builder(
        itemCount: dosenList.length,
        padding: const EdgeInsets.fromLTRB(12, 0, 12, 12),
        itemBuilder: (context, index) {
          final dosen = dosenList[index];
          return Card(
            margin: const EdgeInsets.symmetric(vertical: 6),
            child: ListTile(
              leading: CircleAvatar(child: Text('${dosen.id}')),
              title: Text(dosen.name),
              subtitle: Text(
                '${dosen.username} · ${dosen.email}\n${dosen.address.city}',
              ), // Text
              isThreeLine: true,
              trailing: IconButton(
                icon: const Icon(Icons.save, size: 18),
                tooltip: 'Simpan user ini',
                onPressed: () async {
                  await ref
                      .read(dosenNotifierProvider.notifier)
                      .saveSelectedDosen(dosen);
                  ref.invalidate(savedUsersProvider);
                  if (context.mounted) {
                    ScaffoldMessenger.of(context).showSnackBar(
                      SnackBar(
                        content: Text(
                          '${dosen.username} berhasil disimpan ke local storage',
                        ), // Text
                      ), // SnackBar
                    );
                  }
                },
              ), // IconButton
            ), // ListTile
          ); // Card
        },
      ), // ListView.builder
    ); // RefreshIndicator
  }
}
```

9. **Preview hasil**



10. Buat laporan pengerjaan dalam bentuk **pdf**, ambil sebagian **screen shoot code** dari masing-masing file dan sertakan link **githubnya**.
11. Buat seperti menu **dosen**, untuk menu **mahasiswa**

Selamat Mencoba